

SKILL-03

1.To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Case

```
root@SRIVALLI:~# nano parser.c
root@SRIVALLI:~# ./parser
-bash: ./parser: No such file or directory
root@SRIVALLI:~# nano single_quotes.c
root@SRIVALLI:~# ./single_quotes
-bash: ./single_quotes: No such file or directory
root@SRIVALLI:~# echo 'Hello World'
Hello World
root@SRIVALLI:~# echo '$HOME'
$HOME
root@SRIVALLI:~# echo 'Today is $USER'
Today is $USER
root@SRIVALLI:~# echo ''
```

Report:

In this task, I implemented support for single-quoted strings in the parser. Single quotes were used to preserve the exact text entered by the user without expanding variables. Different test cases, including special characters, spaces, variables, and empty strings, were used to verify the parsing results. This helped in understanding how shells treat single-quoted content as literal text. Single-quoted strings were parsed successfully, and variable expansion was ignored while preserving the original content.

2.To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands

```
root@SRIVALLI:~# echo "Hello World"
Hello World
root@SRIVALLI:~# echo "$HOME"
/root
root@SRIVALLI:~# echo "$USER"
root
root@SRIVALLI:~# echo "User Name: $USER"
User Name: root
root@SRIVALLI:~# echo "Linux Shell Programming"
Linux Shell Programming
root@SRIVALLI:~# echo "Today is $(date)"
Today is Sat Aug 22 13:44:59 UTC 2026
root@SRIVALLI:~# echo "Home Directory: $HOME"
Home Directory: /root
root@SRIVALLI:~# echo ""

root@SRIVALLI:~# cat double_quotes.c
gcc double_quotes.c -o double_quotes
root@SRIVALLI:~# nano double_quotes.c
root@SRIVALLI:~#
root@SRIVALLI:~# ./double_quotes
```

Report:

In this task, I implemented support for double-quoted strings. Double quotes preserved spaces between words while allowing variable expansion. Different test cases involving variables, spaces, and quoted commands were executed to validate the parser output. This helped in understanding how shells process double-quoted strings differently from single-quoted strings. Double-quoted strings were handled successfully, spaces were preserved, and variables were expanded correctly in the output.

3.To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs

```

root@SRIVALLI:~#
root@SRIVALLI:~# nano double_quotes.c
root@SRIVALLI:~# ./double_quotes
-bash: ./double_quotes: No such file or directory
root@SRIVALLI:~# echo "Hello World"
Hello World
root@SRIVALLI:~# echo "$HOME"
/root
root@SRIVALLI:~# echo "$USER"
root
root@SRIVALLI:~# echo "User Name: $USER"
User Name: root
root@SRIVALLI:~# echo "Linux Shell Programming"
Linux Shell Programming
root@SRIVALLI:~# echo "Today is $(date)"
Today is Sat Aug 22 13:44:59 UTC 2026
root@SRIVALLI:~# echo "Home Directory: $HOME"
Home Directory: /root
root@SRIVALLI:~# echo ""

root@SRIVALLI:~# cat double_quotes.c
gcc double_quotes.c -o double_quotes
root@SRIVALLI:~# nano double_quotes.c
root@SRIVALLI:~#
root@SRIVALLI:~# ./double_quotes
-bash: ./double_quotes: No such file or directory
root@SRIVALLI:~# nano escape_sequences.c
root@SRIVALLI:~# ./escape_sequences
-bash: ./escape_sequences: No such file or directory
root@SRIVALLI:~# echo Hello\ World
Hello World
root@SRIVALLI:~# touch My\ File.txt
ls
'My File.txt'          dynamic_memory.c      process_state
OSSP                   escape_sequences.c   process_state.c
SkillWeek1             fork_process         single_quotes.c
command_executor       fork_process.c       test
command_executor.c     history_shell.c      tokenizer.c
command_executor.c.save nano.443.save
double_quotes.c        parser.c
root@SRIVALLI:~# echo \$HOME

```

```
root@SRIVALLI:~# echo \$HOME
$HOME
root@SRIVALLI:~# echo \"Linux\"
"Linux"
root@SRIVALLI:~# echo \\
\
root@SRIVALLI:~# echo \*
echo \?
echo \&
*
?
&
root@SRIVALLI:~# echo My\ Name\ is\ Srivalli
My Name is Srivalli
root@SRIVALLI:~# echo \$USER\ lives\ in\ \$HOME
$USER lives in $HOME
root@SRIVALLI:~# cat escape_sequences.c
gcc escape_sequences.c -o escape_sequences
root@SRIVALLI:~# nano escape_sequences.c
root@SRIVALLI:~# ./escape_sequences
```

Report:

In this task, I implemented support for escape sequences in command parsing. Escaped spaces, special symbols, and reserved characters were handled correctly to preserve the intended input. Various test cases were executed to validate parser output and ensure that complex inputs were processed accurately. This task helped in understanding how shells interpret escaped characters and special symbols. Escape sequences were handled successfully, special characters were preserved correctly, and complex inputs were parsed and validated without errors.

4.To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

```
root@SRIVALLI:~# pstree -p
systemd(1)─agetty(213)
          ─cron(181)
          ─dbus-daemon(182)
          ─init-systemd(Ub(2)─SessionLeader(320)─Relay(322)(321)─ba
                                ─init(6)─{init}(7)
                                ─login(323)─bash(391)
                                ─{init-systemd(Ub}(8)
          ─rsyslogd(214)─{rsyslogd}(235)
                        ─{rsyslogd}(236)
                        ─{rsyslogd}(238)
          ─systemd(366)─(sd-pam)(367)
          ─systemd-journal(55)
          ─systemd-logind(192)
          ─systemd-resolve(117)
          ─systemd-timesyn(118)─{systemd-timesyn}(150)
          ─systemd-udev(103)
          ─unattended-upgr(222)─{unattended-upgr}(274)
```

```

Reading state information... Done
psmisc is already the newest version (23.7-1build1).
0 upgraded, 0 newly installed, 0 to remove and 10 not upgraded.
root@SRIVALLI:~# pstree -p
systemd(1)─agetty(213)
          └─cron(181)
          └─dbus-daemon(182)
          └─init-systemd(Ub(2))─SessionLeader(320)─Relay(322)(321)─b
                                └─init(6)─{init}(7)
                                └─login(323)─bash(391)
                                └─{init-systemd(Ub)}(8)
          └─rsyslogd(214)─{rsyslogd}(235)
                        └─{rsyslogd}(236)
                        └─{rsyslogd}(238)
          └─systemd(366)─(sd-pam)(367)
          └─systemd-journal(55)
          └─systemd-logind(192)
          └─systemd-resolve(117)
          └─systemd-timesyn(118)─{systemd-timesyn}(150)
          └─systemd-udev(103)
          └─unattended-upgr(222)─{unattended-upgr}(274)

root@SRIVALLI:~# top
top - 14:07:02 up 26 min,  1 user,  load average: 0.00, 0.00, 0.00
Tasks: 21 total,  1 running, 20 sleeping,  0 stopped,  0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0
MiB Mem :  7784.6 total,  7139.3 free,   517.1 used,   278.4 buff/cach
MiB Swap: 2048.0 total, 2048.0 free,    0.0 used.  7267.6 avail Mem

  PID USER      PR  NI   VIRT    RES    SHR S  %CPU  %MEM    TIME+
   1 root        20   0   21688   13088   9744 S   0.0   0.2   0:00.95
   2 root        20   0    3180    2208   2072 S   0.0   0.0   0:00.01
   6 root        20   0    3196    2132   2008 S   0.0   0.0   0:00.00
  55 root        19  -1  50368  16020 14920 S   0.0   0.2   0:00.33
 103 root        20   0   25280    6612   5176 S   0.0   0.1   0:00.50
 117 systemd+    20   0   21344   13156 11032 S   0.0   0.2   0:00.19
 118 systemd+    20   0   91036    7992   7024 S   0.0   0.1   0:00.15
 181 root        20   0    4244    2684   2432 S   0.0   0.0   0:00.02
 182 message+    20   0    9540    5392   4700 S   0.0   0.1   0:00.13
 192 root        20   0   17980    8852   7812 S   0.0   0.1   0:00.17
 213 root        20   0    3124    1968   1836 S   0.0   0.0   0:00.00
 214 syslog     20   0  222516   6048   4680 S   0.0   0.1   0:00.15

```

Report:

In this task, I created child processes using `fork()` and executed programs using the `exec()` system call. Command arguments were passed to the child process, while the parent process waited for the child to complete execution. Different commands were tested, and error handling was verified using invalid commands. Child processes were created successfully, commands were executed correctly, and the parent process managed child execution properly. Error conditions were also handled successfully.

